

BAZE PODATAKA ZAVRŠNI

Uvod: osnovni termini i koncepti

1. ANSI/SPARC arhitektura baze podataka.

• Fizički nivo

Bavi se organizacijom i razmještanjem podataka na fizičkom medijumu u cilju što efikasnijeg pristupa podacima u BP.

Korisnici ne moraju da znaju detalje interne fizičke organizacije podataka, koji su u fokusu timova za razvoj DBMS-a.

Podaci stvarno egzistiraju samo na fizičkom nivou!

Predstava podataka na svakom nivou apstrakcije opisuje se odgovarajućom šemom/modelom.

• Logički nivo

Opisuje sveukupnu logičku strukturu podataka i veza između njih.

Opis može da bude na:

- Konceptualnom nivou
- Prezentacionom nivou (implementacionom nivou) – opisuje koji podaci (određenih tipova) se smjštaju u BP i kakve su veze između podataka.

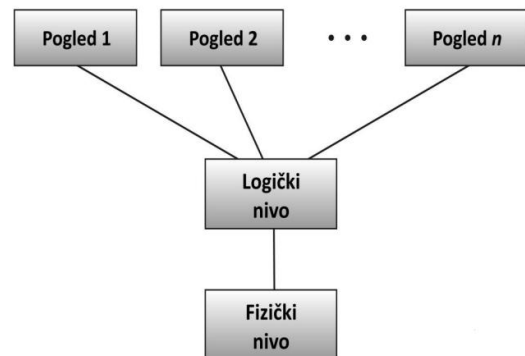
• Nivo pogleda

Korisnik vidi samo specifičan podskup sveukupne logičke šeme – alternativno se naziva **lokalni logički nivo**.

Korisniku su dostupni samo podaci koji su mu potrebni (kroz aplikativni program).

Pogledi omogućavaju skrivanje pojedinih informacija od korisnika iz sigurnosnih razloga.

ANSI/SPARC arhitektura BP



2. Upravljanje smještanjem podataka u DBMS.

Podsistem za smještanje podataka (Storage management) obezbjeđuje interfejs između podataka smještenih na fizičkom mediju i procesora upita koji konvertuje upite iz aplikativnih programa/DML iskaza u instrukcije niskog nivoa.

Storage manager konvertuje instrukcije niskog nivoa dobijene od procesora upita u operacije niskog nivoa za efikasan pristup i manipulaciju podacima u bazi.

Storage manager je odgovoran za sljedeće zadatke:

- interakcija sa *file manager*-om (npr. alokacija prostora na disku)

- efikasno smještanje, pretraživanje i ažuriranje podataka, upravljanje transakcijama, konkurentno izvršavanje, autorizaciju i integritet.

Komponente *storage manager*-a:

- upravljanje baferom (*buffer manager*)
- komponenta za upravljanje transakcijama (*transaction manager*)
- komponenta za autorizaciju i osiguranje integriteta (*authorisation and integrity manager*)

Kreira i radi sa sljedećim strukturama:

- podaci / datoteke
- rječnik podataka (*data dictionary*)
- indeksne strukture i
- statistike

3. Osnovne komponente i opis funkcionisanja sistema za upravljanje bazama podataka.

Osnovne komponente sistema za upravljanje bazama podataka su:

- **procesor upita** – koji procesira DDL i DML iskaze i prosljeđuje ih podsystemu za upravljanje smještanjem podataka.
- **podsystem za upravljanje smještanjem podataka** (*storage manager*) – implementira efikasan fizički pristup podacima na disku i manipulaciju podacima.

Proces izvršavanja upita se odvija na sljedeći način:

1. Kompajlirani *ad hoc* upiti, kao i predefinisani upiti kodovani u programskim aplikacijama prosljeđuju se **optimizatoru**.

2. Optimizator kreira plan izvršenja upita sa ciljem postizanja najboljih performansi odziva i koristi odgovarajuće algoritme i konsultuje podatke o raspoloživim pristupnim putevima, te statističke podatke o prethodnim upitima za kreiranje plana izvršenja.

3. **Optimizator** prosljeđuje optimizovani plan izvršenja **run-time** procesoru, koji prima i izvršava zahtjeve za pretraživanje i ekstrakciju ili ažuriranje podataka u bazi u formi instrukcija niskog nivoa.

4. Arhitektura sistema sa bazama podataka.

Aplikacija sa bazom podataka se sastoji od baze podataka i programa za unos i ažuriranje, obradu, pretraživanje i prezentaciju podataka.

Arhitektura aplikacije može biti centralizovana ili klijent-server.

Centralizovana arhitektura

Ovo je tipična arhitektura u početku razvoja sistema sa bazama podataka.

Centralni računar (mainframe) služi za cjelokupnu obradu podataka.

Neinteligentni terminali služe samo za ulaz / izlaz podataka.



Klijent-server arhitektura

Klijentska komponenta aplikacije se izvršava na jednom računarskom sistemu kao **klijentski proces**, zahtjeva servis od druge aplikativne komponente koja se izvršava na drugom računarskom sistemu kao **serverski proces**.



Dvoslojna klijent-server arhitektura

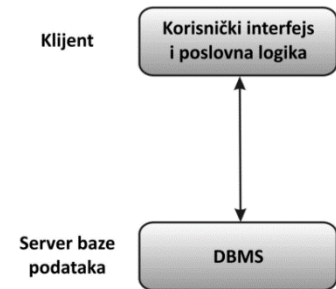
Najjednostavnija implementacija:

- klijent: specifične aplikativne funkcije (komunikacija sa korisnikom i poslovna logika)
- server: DBMS + baza podataka

Za povezivanje aplikativnih programa i DBMS koriste se odgovarajući API, npr. ODBS ili JDBC.

Na klijentskoj strani aplikativni programi mogu specificovati upite/operacije nad podacima u BP sistema za upravljanje bazom podataka na serverskoj strani.

DBMS obezbjeđuje izvršavanje upita i vraćanje rezultata aplikativnom programu.

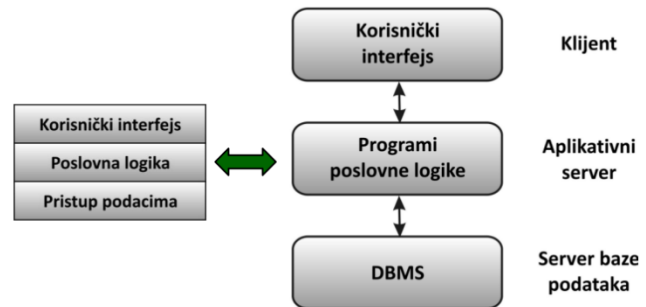


Troslojna klijent-server arhitektura

Ima značajnu prednost u tome što je moguće raditi modifikacije u jednom sloju bez potrebe modifikacija u drugim slojevima.

Aplikativni sistemi sa BP sadrže tri logičke cjeline (sloja):

- komunikacija sa korisnikom (korisnički interfejs)
- poslovna/aplikativna logika
- pristup bazi podataka



Svaku logičku cjelinu moguće je implemetirati odgovarajućim slojem – modulom na (tipično) različitim mašinama

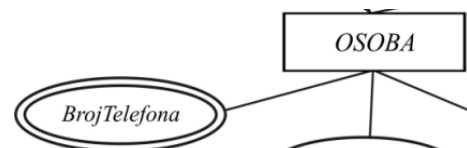
- korisnički interfejs na klijentskom računaru
- poslovna logika na aplikativnom serveru
- pristup podacima na serveru BP

Konceptualno modelovanje

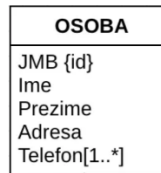
5. Višeznačni atributi. Modelovanje višeznačnih atributa u MOV i u UML.

Višeznačni (multi-valued) atributi mogu imati više od jedne vrijednosti za neki entitet (npr. brojevi telefona neke osobe).

U MOV dijagramu se predstavljaju **duplom elipsom**.



U UML dijagramu višeznačni atributi se predstavljaju tako što se za dati atribut specifikuje multiplikativnost pored navedenog atributa unutar srednjih zagrada.



6. Ograničenja učešća entiteta u vezi. Objasniti i primjerom ilustrovati reprezentaciju ograničenja učešća entiteta u vezi u MOV dijagramu i UML dijagramu klasa.

Ograničenje učešća (participacije) entiteta u vezi specifikuje, za određeni tip veze, minimalan broj veza u kojima neki entitet odnosnog entitetskog tipa (skupa) mora da učestvuje.

Postoje dvije vrste ograničenja na učešće entiteta u vezi: **totalno** i **parcijalno**.

Totalno učešće entitetskog tipa u tipu veze:

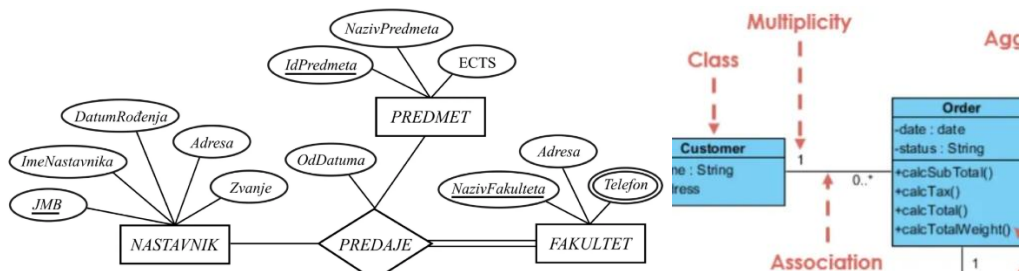
- Svaki entitet tog entitetskog tipa mora učestvovati u barem jednoj vezi iz veznog skupa.
- Ograničenje se naziva i **egzistencijalna zavisnost**.

Parcijalno učešće entitetskog tipa u tipu veze:

- **Mogu postojati neki entiteti datog entitetskog tipa bez učešća u vezama iz veznog skupa.**

U MOV dijagramu totalno učešće nekog ET u vezi se označava **duplim linijama**, a parcijalno učešće se označava **jednostrukim linijama**.

U UML dijagramu klasa ograničenje učešća se modeluje pomoću multiplikativnosti krajeva asocijacije. Ako se stavi 0..n, 0..* ili 0..1 tada se označava parcijalno učešće, a ako se stavi 1..n, 1..* ili 1 tada se označava totalno učešće.



7. Ključevi veznih tipova/skupova.

Kombinacija primarnih ključeva participirajućih entitetskih tipova (skupova) u tipu veze (veznom skupu), čini **superključ tipa veze** (veznog skupa).

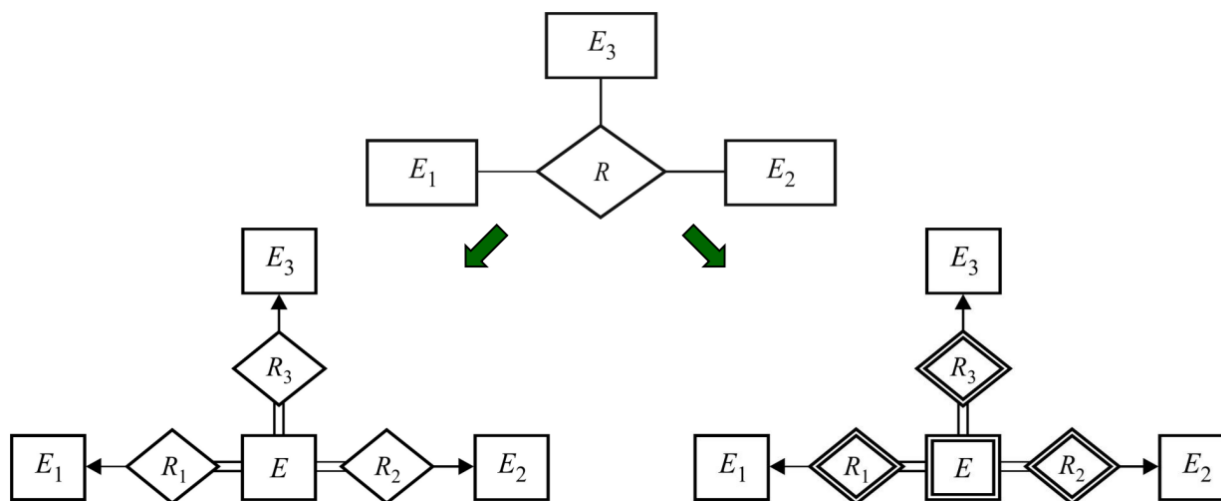
Samo jedna veza može biti sa vrijednošću superključa koja je kombinacija vrijednosti primarnih ključeva entiteta koji učestvuju u vezi.

Kod identifikacije **kandidatskog/primarnog ključa veznog tipa** (skupa), neophodno je uzeti u razmatranje kardinalnost mapiranja odnosnog veznog tipa (skupa).

- Ako je kardinalnost mapiranja M:M, tada je veza identifikovana sa unijom oba primarna ključa.
 - Jedan nastavnik može da predaje više predmeta, a isti predmet može da predaje više nastavnika. $PK(NASTAVNIK) \cup PK(PREDMET)$ je superključ, a primarni ključ $PK(PREDAJE) = PK(NASTAVNIK) \cup PK(PREDMET)$.
- Ako je kardinalnost mapiranja 1:M ili M:1, tada je veza identifikovana ključem entiteta sa strane M.
 - Jedan fakultet može biti matičan za više predmeta, a za neki predmet je matičan samo jedan fakultet. $PK(FAKULTET) \cup PK(PREDMET)$ je superključ, a primarni ključ $PK(MATICAN_ZA) = PK(PREDMET)$.
- Ako je kardinalnost mapiranja 1:1, tada se može uzeti bilo koji primarni ključ da identifikuje vezu.
 - Jedan nastavnik može da rukovodi samo jednim fakultetom, i jedan fakultet može da ima samo jednog nastavnika rukovodioca – dekana. $PK(FAKULTET) \cup PK(NASTAVNIK)$ je superključ, a primarni ključ $PK(RUKOVODI)$ može biti ili $PK(FAKULTET)$ ili $PK(NASTAVNIK)$.

8. Objasniti i primjerima ilustrovati mogućnost za konverziju MOV sa n-arnim veznim tipom u MOV sa ekvivalentnim binarnim veznim tipom, a zatim napisati DDL skripte za generisanje korespodentnih šema relacione baze podataka.

Svaka n-arna veza ($n > 2$) se može zamijeniti ekvivalentnim skupom binarnih veza. Npr: ternarna veza R entitetskih tipova E1, E2 i E3 se može zamijeniti na jedan od sljedeća dva načina.



Na lijevoj slici E je vještački entitetski tip koji kao atribut ima vještački kreiran PK (surogat ključ) + atribut veznog tipa R.

Na desnoj slici E je slab entitetski tip kojeg identifikuju jaki entitetski tipovi E1, E2 i E3, a primarni ključ je $PK(E) = PK(E1) \cup PK(E2) \cup PK(E3)$. Ukoliko R ima opisne attribute oni postaju atributi slabog entitetskog tipa E.

```
create table E1 (  
    e1_id int primary key  
);  
  
create table E2 (  
    e2_id int primary key  
);  
  
create table E3 (  
    e3_id int primary key  
);  
  
create table E (  
    e_id int primary key,  
    e1_id int not null,  
    e2_id int not null,  
    e3_id int not null,  
    constraint fk_e_e1 foreign key (e1_id) references E1(e1_id),  
    constraint fk_e_e2 foreign key (e2_id) references E2(e2_id),  
    constraint fk_e_e3 foreign key (e3_id) references E3(e3_id)  
/*  
    atributi veznog tipa  
    ...  
*/  
);
```

```
create table E1 (  
    e1_id int primary key  
);  
  
create table E2 (  
    e2_id int primary key  
);  
  
create table E3 (  
    e3_id int primary key  
);  
  
create table E (  
    e1_id int,  
    e2_id int,  
    e3_id int,  
    primary key (e1_id, e2_id, e3_id),  
    constraint fk_e_e1 foreign key (e1_id) references E1(e1_id),  
    constraint fk_e_e2 foreign key (e2_id) references E2(e2_id),  
    constraint fk_e_e3 foreign key (e3_id) references E3(e3_id)  
/*  
    atributi veznog tipa  
    ...  
*/  
);
```

9. Objasniti i konkretnim primjerom ilustrovati modelovanje specijalizacije u MOV.

U procesu projektovanja „od vrha na dole“ (*top-down*) u određenim slučajevima možemo uočiti da u okviru nekog entitetskog skupa postoje grupe entiteta koje se međusobno razlikuju.

Pored zajedničkih karakteristika, mogu se uočiti i specifične karakteristike pojedinih podgrupa – te karakteristike mogu se izraziti dodatnim specifičnim atributima za pojedine podgrupe.

Podgrupe (entitetski tipovi „nižeg nivoa“) mogu učestvovati u specifičnim vezama, u kojima ne mogu participirati entiteti koji nisu članovi podgrupe.

Niži entitetski tip nasljeđuje sve atribute i participacije u vezama višeg entitetskog tipa sa kojim je povezan.

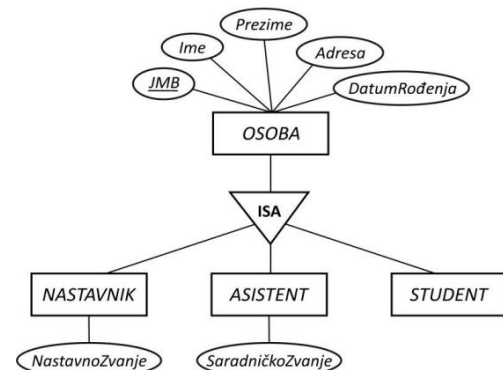
Veza između entitetskog tipa višeg nivoa i entitetskih tipova nižeg nivoa se prikazuje kao trougaona komponenta nazvana ISA.

Inverzna pojava specijalizacije je generalizacija (bilo koji pojam da koristimo pravilno je). U procesu projektovanja „od dna prema vrhu“ (*bottom-up*) u određenim situacijama možemo uočiti da neki entitetski skupovi imaju određene zajedničke karakteristike, na bazi kojih se ti entitetski skupovi mogu kombinovati u jedan entitetski skup „višeg nivoa“.

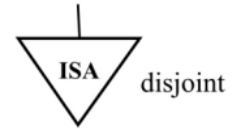
Kriterijumi za generalizaciju/specijalizaciju (može postojati više specijalizacija entitetskog skupa na bazi različitih kriterijuma):

a. Objasniti i konkretnim primjerima ilustrovati projektna ograničenja koja se odnose na specijalizaciju u MOV dijagramu.

- Ograničenje koje se odnosi na to koji entiteti mogu pripadati datom entitetskom skupu na nižem nivou:



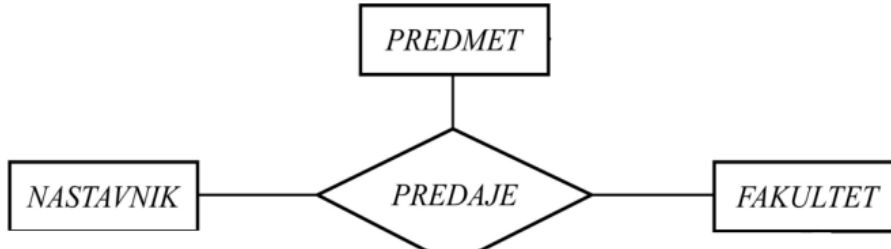
- **uslovno-definisano**
- **korisnički-definisano**
- Ograničenje koje specifikuje **da li neki entitet superklase može da bude član više potklasa:**
 - **disjunktno** (*disjoint*) – entitet može da pripada samo jednoj potklasi, na MOV dijagramu se uz trougao dodaje ključna riječ „disjoint“
 - **preklapajuće** (*overlapping*) – entitet može da bude član više od jedne potklase
- **Ograničenje kompletnosti** – specifikuje da li neki entitet superklase mora pripadati najmanje jednoj potklasi u okviru specijalizacije:
 - **totalna specijalizacija** – entitet superklase mora pripadati najmanje jednoj potklasi, u MOV dijagramu se dodaje dodatna linija
 - **parcijalna specijalizacija** – entitet superklase ne mora pripadati nekoj potklasi



10. Objasniti i primjerom ilustrovati koncept agregacije u MOV.

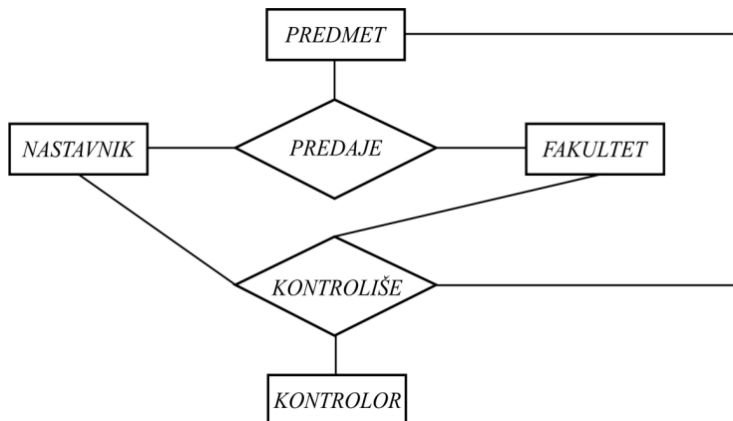
Generalno, agregacija je koncept kojim se kreira složeni atribut, kompozitni objekat od komponentnih objekata, tj. u slučaju MOV dijagrama agregacija predstavlja apstrakciju kojom se veze tretiraju kao entiteti višeg nivoa.

Posmatrajmo primjer:



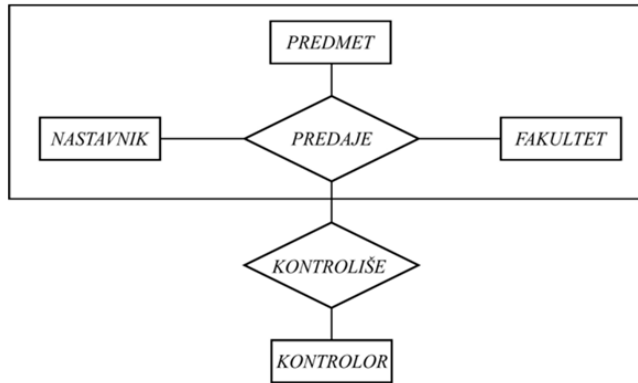
Pretpostavimo da želimo da dodamo kontrolore za nadzor nad izvođenjem nastave koju izvršavaju određeni nastavnici na nekom predmetu i nekom fakultetu.

Dobijamo sljedeći dijagram:



Vezni tipovi KONTROLIŠE i PREDAJE reprezentuju informacije koje se preklapaju. Svaka veza tipa KONTROLIŠE sadrži odgovarajuću vezu tipa PREDAJE, ali sve veze tipa PREDAJE ne moraju imati odgovarajuću vezu tipa KONTROLIŠE. Zbog toga ne možemo kombinovati ova dva tipa u jedan.

Agregacija nam omogućava da vezu tipa PREDAJE modeluje kao apstraktni entitet višeg nivoa, pa vezu KONTROLIŠE pretvorimo u binarnu vezu sa tim apstraktnim tipom PREDAJE.



Relacioni model

11. Definirati pojmove superključ, kandidatski ključ i primarni ključ te ih ilustrovati na konkretnom primjeru jedne relacione šeme.

Neka je R relaciona šema definisana nad atributima $A_1, A_2, \dots, A_n$: $R = A_1, A_2, \dots, A_n$. Skup atributa relacione šeme R se označava kao $\text{attrs}(R)$.

Neka je $K \subseteq \text{attrs}(R)$.

K je **superključ** relacione šeme R ako su vrijednosti K dovoljne da **jedinstveno identifikuju svaku torku za svaku moguću relaciju** $r(R)$, tj. ako za svaki par torki $t_1 \in r$ i $t_2 \in r$ vrijedi:

$$t_1 \neq t_2 \Rightarrow t_1[K] \neq t_2[K]$$

K je **kandidatski ključ** ako je K superključ i nijedan pravi podskup atributa od K nije superključ.

Primarni ključ je jedan od kandidatskih ključeva.

Primjer: Neka je data relaciona šema $PREDMET(IdPredmeta, NazivPredmeta, ECTS)$. Superključ na šemi $PREDMET$ je: $SK(PREDMET) = \{IdPredmeta\}, \{IdPredmeta, NazivPredmeta\}, \dots$ Kandidatski ključ za relacionu šemu $PREDMET$ je $\{IdPredmeta\}$, a budući da je jedinstven to je i primarni ključ, dakle $PK(PREDMET) = \{IdPredmeta\}$.

12. Ograničenja po referencijalnom integritetu.

Neka su $r_1(R_1)$ i $r_2(R_2)$ relacije na šemama R_1 i R_2 sa primarnim/kandidatskim ključevima K_1 i K_2 , respektivno.

$FK \subset \text{attrs}(R_1)$ je **strani ključ** u R_1 koji **referencira** primarni/kandidatski ključ K_2 u R_2 ako:

- atributi stranog ključa FK imaju isti domen kao korespondentni atributi primarnog/kandidatskog ključa K_2 u R_2 , i
- za svaku torku $t_1 \in r_1$
 - o ili mora postojati torka $t_2 \in r_2$ takva da je $t_2[K_2] = t_1[FK]$
 - o ili je vrijednost $t_1[FK] = \text{null}$

Relacija r_1 (relaciona šema R_1) naziva se **raferencirajuća** relacija (šema).

Relacija r_2 (relaciona šema R_2) naziva se **referencirana** relacija (šema).

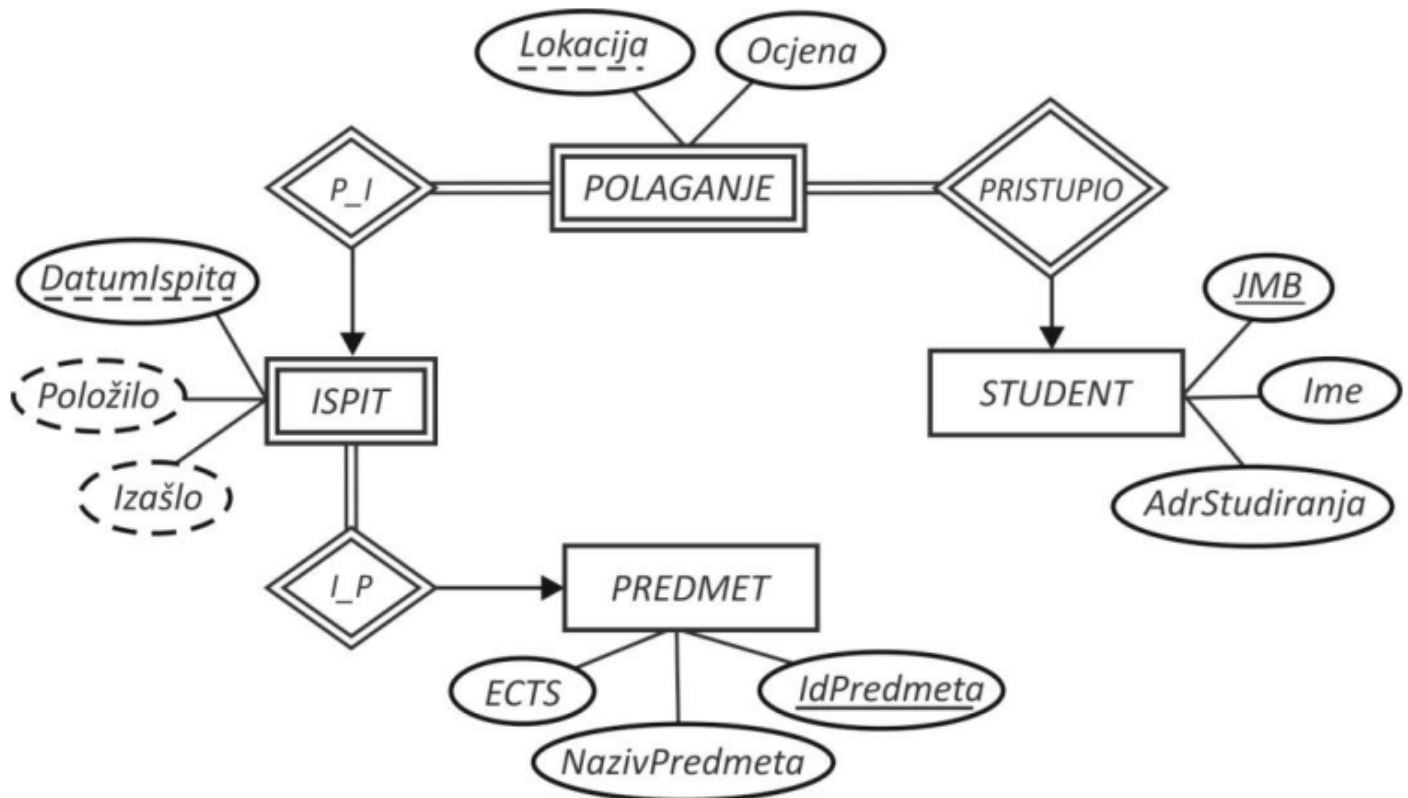
Navedeni uslovi specificuju **ograničenje po referencijalnom integritetu** na šemama R_1 i

R_2 .

13. Objasniti i konkretnim primjerom ilustrirati mapiranje slabog entitetskog tipa iz konceptualnog modela u relacioni model. Napisati odgovarajuće SQL upite.

Ako mapiramo slabi entitetski tip Y sa diskriminatorom D i jakim entitetskim tipom X koji identifikuje slabi entitetski tip Y u relacionu šemu R, tada:

- atributi(Y) $\rightarrow$ atributi(R)
- $PK(X), D(Y) \rightarrow PK(R) = PK(X) \cup D(Y)$



Primjer: Za MOV sa slike slabi entitetski tipovi su ISPIT i POLAGANJE. Slabi ET ISPIT je identifikovan od strane jakog ET PREDMET i ima diskriminator DatumIspita, pa će na osnovu definisanog pravila relaciona šema biti ISPIT(IdPredmeta, DatumIspita). Slabi ET POLAGANJE je identifikovan od strane jakog ET STUDENT i slabog ET ISPIT, pa je relaciona šema sljedeća: POLAGANJE(IdPredmeta, DatumIspita, JMB, Lokacija, Ocjena).

SQL upiti su:

```
create table Student (  
    JMBG char(13) primary key,  
    Ime varchar(100) not null,  
    AdrStudiranja varchar(100) not null  
);  
  
create table Predmet (  
    IdPredmeta varchar(10) primary key,  
    NazivPredmeta varchar(100) not null,  
    ECTS int not null  
);  
  
create table Ispit (  
    IdPredmeta int,  
    DatumIspita date,  
    constraint fk_ispit_predmet foreign key (IdPredmeta) references Predmet(IdPredmeta),  
    primary key (IdPredmeta, DatumIspita)  
);  
  
create table Polaganje (  
    IdPredmeta varchar(10),  
    DatumIspita date,  
    JMBG char(13),  
    Lokacija varchar(100),  
    Ocjena int not null,  
    constraint fk_polaganje_ispit foreign key (IdPredmeta, DatumIspita) references Ispit(IdPredmeta, DatumIspita),  
    constraint fk_polaganje_student foreign key (JMBG) references Student(JMBG),  
    primary key (JMBG, IdPredmeta, DatumIspita, Lokacija)  
);
```

14. Objasniti i konkretnim primjerima ilustrovati mogućnost za mapiranje veze tipa 1:M iz konceptualnog modela u relacionu šemu. Za svaki slučaj napisati odgovarajuću DDL skriptu za generisanje korespondentne šeme relacione baze podataka (MySQL).

Mapiranje konceptualnog modela zavisi od učešća entiteta na strani M, tj. da li je totalno ili parcijalno učešće.

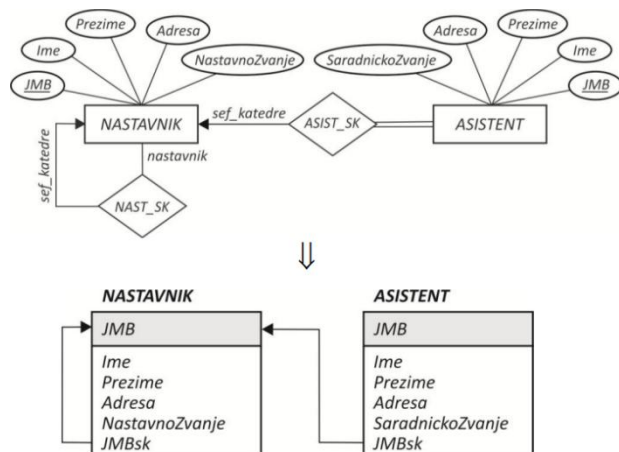
Totalno učešće entitetskog tipa sa M strane

U relacionu šemu T, koja je generisana za tip sa M strane, dodaju se atributi PK tipa sa strane 1 (pristup korištenjem stranog ključa).

Dodatni atributi čine strani ključ u šemi T, koji referencira relacionu šemu koja korespondira tipu sa strane 1.

Ukoliko tip veze ima proste opisne attribute, oni se takođe uključuju u relacionu šemu T, kao i proste komponente kompozitnih atributa tipa veze.

Torke relacije, definisane na generisanoj šemi T, reprezentuju korespondentne attribute entiteta tipa sa M strane i vezu svakog od tih entiteta sa jednim entitetom sa strane 1.



Parcijalno učešće entitetskog tipa sa M strane

Postoje dva načina da se izvrši ovo mapiranje:

1. Isto kao i kada je totalno učešće – obzirom na to da sada neki entiteti sa strane M ne učestvuju u vezi, torke korespodentne relacije imaće vrijednost null na atributima stranog ključa i opisnim atributima veze, jer za te entitete ne postoji veza sa odnosnim entitetom tipa sa strane 1.

2. Vezni tip se mapira u veznu šemu – kreira se posebna vezna relaciona šema za dati tip veze, koja uključuje: atribute PK oba entitetska tipa koji učestvuju u vezi i sve proste attribute i proste komponente kompozitnih atributa tipa veze. Atributi PK svakog tipa koji učestvuje u vezi, u generisanoj šemi čine FK koji referencira šemu u koju je mapiran odnosni entitetski tip. PK generisane relacione šeme čine atributi PK entitetskog tipa sa M strane.

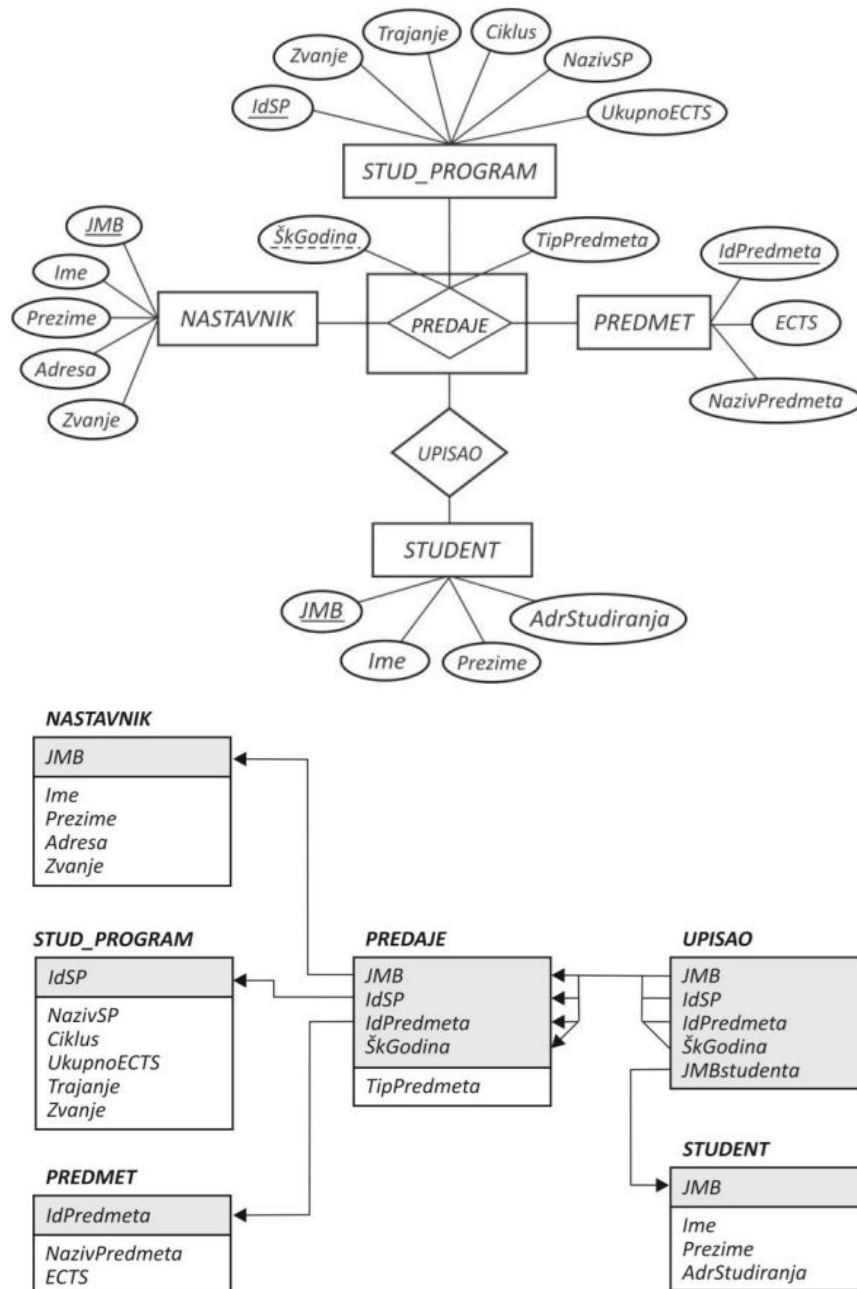
DDL skripta za generisanje korespodentne relacione šeme baze podataka:

```
create table Nastavnik (  
    JMB char(13) primary key,  
    Ime varchar(100) not null,  
    Prezime varchar(100) not null,  
    Adresa varchar(100) not null,  
    NastavnoZvanje varchar(100) not null,  
    JMBsk char(13),  
    constraint fk_nastavnik_sef_katedre foreign key (JMBsk) references Nastavnik(JMB)  
);  
  
create table Asistent (  
    JMB char(13) primary key,  
    Ime varchar(100) not null,  
    Prezime varchar(100) not null,  
    Adresa varchar(100) not null,  
    SaradnickoZvanje varchar(100) not null,  
    JMBsk char(13) not null,  
    constraint fk_asistent_nastavnik foreign key (JMBsk) references Nastavnik(JMB)  
);  
  
create table Nastavnik (  
    JMB char(13) primary key,  
    Ime varchar(100) not null,  
    Prezime varchar(100) not null,  
    Adresa varchar(100) not null,  
    NastavnoZvanje varchar(100) not null  
);  
  
create table SefKatedre (  
    JMB char(13) primary key,  
    JMBsk char(13) not null,  
    constraint fk_sef_katedre_nastavnik foreign key (JMB, JMBsk) references Nastavnik(JMB, JMB)  
);  
  
create table Asistent (  
    JMB char(13) primary key,  
    Ime varchar(100) not null,  
    Prezime varchar(100) not null,  
    Adresa varchar(100) not null,  
    SaradnickoZvanje varchar(100) not null,  
    JMBsk char(13) not null,  
    constraint fk_asistent_nastavnik foreign key (JMBsk) references Nastavnik(JMB)  
);
```

15. Objasniti i primjerom ilustrovati mapiranje agregacije iz MOV u relacioni model, te specificovati DDL skriptu.

Veza koja se tretira kao agregacija se mapira u relacionu šemu po pravilima za mapiranje binarnog ili n-arnog tipa veze.

Kreira se relaciona šema koja uključuje attribute PK tipova koji učestvuju u agregaciji i proste attribute tipa veze. PK nove šeme čine PK šema koje učestvuju u agregaciji i diskriminator agregacije.



DDL skripta:

```
create table Nastavnik (  
    JMB char(13) primary key,  
    Ime varchar(100) not null,  
    Prezime varchar(100) not null,  
    Adresa varchar(100) not null,  
    Zvanje varchar(100) not null  
);  
  
create table Stud_program (  
    IdSP varchar(10) primary key,  
    Zvanje varchar(100) not null,  
    Trajanje int not null,  
    Ciklus int not null,  
    NazivSP varchar(100) not null,  
    UkupnoECTS int not null  
);  
  
create table Predmet (  
    IdPredmeta varchar(10) primary key,  
    NazivPredmeta varchar(100) not null,  
    ECTS int not null  
);  
  
create table Student (  
    JMB char(13) primary key,  
    Ime varchar(100) not null,  
    Prezime varchar(100) not null,  
    AdresaStudiranja varchar(100) not null  
);  
  
create table Predaje (  
    JMB char(13) not null,  
    IdPredmeta varchar(10) not null,  
    IdSP varchar(10) not null,  
    SkGodina int not null,  
    TipPredmeta varchar(100) not null,  
    primary key (JMB, IdPredmeta, IdSP, SkGodina),  
    constraint fk_predaje_nastavnik foreign key (JMB) references Nastavnik(JMB),  
    constraint fk_predaje_predmet foreign key (IdPredmeta) references Predmet(IdPredmeta),  
    constraint fk_predaje_stud_program foreign key (IdSP) references Stud_program(IdSP)  
);  
  
create table Upisao (  
    JMB char(13) not null,  
    IdSP varchar(10) not null,  
    IdPredmeta varchar(10) not null,  
    SkGodina int not null,  
    JMBStudenta char(13) not null,  
    primary key (JMB, IdSP, IdPredmeta, SkGodina, JMBStudenta),  
    constraint fk_upisao_student foreign key (JMBStudenta) references Student(JMB),  
    constraint fk_upisao_predaje foreign key (JMB, IdPredmeta, IdSP, SkGodina)  
    references Predaje(JMB, IdPredmeta, IdSP, SkGodina)  
);
```

16. Objasniti i konkretnim primjerom ilustrovati mapiranje unije iz MOV u relacioni model.

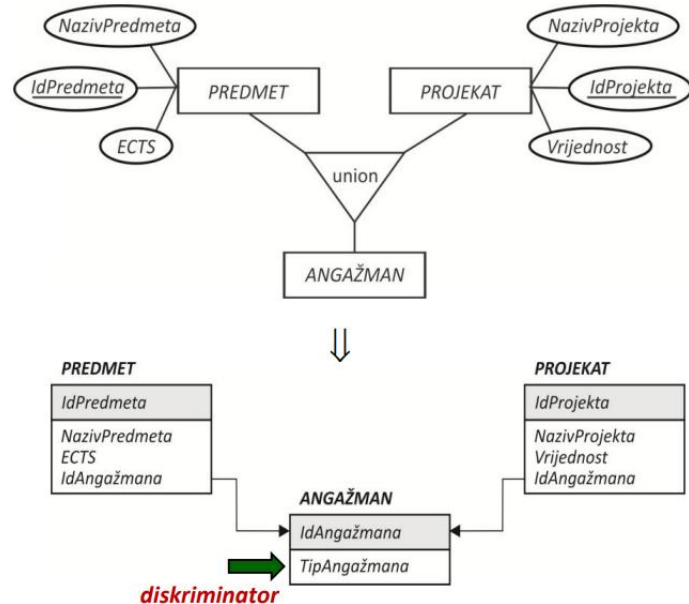
Unija sadrži entitete superklasa različitih tipova. Superklase imaju različite PK. Za uniju treba kreirati posebnu šemu koja uključuje: specifični vještački (surogat) ključ VK i sve specifične proste attribute unije (ako ih unija ima).

PK(UNIJA) = VK

Dodatno, u šeme koje korespondiraju superklasama treba dodati atribut VK – to je FK koji referencira šemu koja korespondira uniji.

Informacije o nekom entitetu iz unije mogu se naći u odgovarajućoj superklasi, preko vrijednosti stranog ključa VK.

Ako neka torka relacije, koja korespondira entitetskom skupu superklase, ima vrijednost null na atributu VK, onda to znači da odnosni entitet superklase nije član unije.



Relaciona algebra

17. Osnovni operatori relacione algebre.

Postoji 6 osnovnih operatora relacione algebre:

1. Selekcija

Notacija operacije selekcije je: $\sigma_P(r)$.

P se naziva predikat ili uslov selekcije.

P je formula u propozicionom računu koja se sastoji od članova povezanih logičkim operatorima: $\wedge$ (and), $\vee$ (or), $\neg$ (not)

Svaki član je oblika **<attribute> op (<attribute> ili <constant>)**, gdje je op jedan od: =, $\neq$, >, $\geq$, <, $\leq$.

Operacija selekcije se može definisati kao: $\sigma_P(r) = \{ t \mid t \in r \wedge P(t) \}$

2. Projekcija

Notacija operacije projekcije je: $\pi_{\langle \text{lista atributa} \rangle}(r)$, gdje < lista atributa > sadrži attribute relacije r koji treba da se pojave u rezultatnoj relaciji.

Rezultat je definisan kao relacija dobijena (iz polazne relacije) brisanjem atributa (kolona tabele) koji nisu navedeni u listi atributa.

Duple torke se eliminišu iz rezultata s obzirom na to da su relacije skupovi.

Relacija r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

$\sigma_{A=B \wedge D > 5}(r)$

A	B	C	D
α	α	1	7
β	β	23	10

Relacija r

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

$\pi_{A,C}(r)$

A	C
α	1
α	1
β	1
β	2

A	C
α	1
β	1
β	2

3. Unija

Notacija operacije unije je: $r \cup s$.

Definiše se kao: $r \cup s = \{t \mid t \in r \vee t \in s\}$.

Da bi operacija $r \cup s$ bila validna, moraju biti ispunjeni sljedeći uslovi:

1. r, s moraju imati isti stepen / broj atributa ili arnost
2. Domeni atributa moraju biti kompatibilni.

Npr. Lista matičnih brojeva osoba koje su nastavnici ili studenti se dobija sljedećom operacijom: $\pi_{JMB}(\text{nastavnik}) \cup \pi_{JMB}(\text{student})$

4. Razlika

Notacija operacije razlike je: $r - s$.

Definiše se kao: $r - s = \{t \mid t \in r \wedge t \notin s\}$.

Operacija razlike se mora izvoditi između kompatibilnih relacija:

- r i s moraju imati isti stepen / broj atributa
- Korespondentni domeni atributa relacije r i s moraju biti kompatibilni

5. Dekartov proizvod

Notacija operacije Dekartovog proizvoda je: $r \times s$.

Definiše se kao: $r \times s = \{tu \mid t \in r \wedge u \in s\}$.

Šema $R \times S$ rezultatne relacije dobija se konkatencijom šema R i S , odnosno, za šeme $R = (A_1, A_2, \dots, A_n)$ i $S = (B_1, B_2, \dots, B_m)$ dobijamo $R \times S = (A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$.

Pretpostavljamo da su atributi $r(R)$ i $s(S)$ disjunktni ($\text{attrs}(R) \cap \text{attrs}(S) = \emptyset$).

Ukoliko atributi $r(R)$ i $s(S)$ nisu disjunktni, tada moramo primjeniti preimenovanje.

6. Preimenovanje

Operacija preimenovanja omogućava imenovanje rezultata izraza relacione algebre, nakon čega se rezultat referencira korištenjem novog imena.

Omogućava i da jednoj relaciji dodijelimo više od jednog imena.

Primjer: $\rho_X(E)$ vraća izraz E pod imenom X .

Ako izraz E relacione algebre ima n atributa, tada $\rho_{X(A_1, A_2, \dots, A_n)}(E)$ vraća rezultat izraza E kao relaciju sa imenom X , koja ima (preimenovane) attribute $A_1, A_2, \dots, A_n$.

r		s	
A	B	A	B
α	1	α	2
α	2	β	3
β	1		

$r \cup s$	
A	B
α	1
α	2
β	1
β	3

r		s	
A	B	A	B
α	1	α	2
α	2	β	3
β	1		

$r - s$	
A	B
α	1
β	1

r		s		
A	B	C	D	E
α	1	α	10	a
β	2	β	10	a
		β	20	b
		γ	10	b

$r \times s$				
A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

fakultet

NazivFak	AdresaFak
Elektrotehnički	Patre 5 Banjaluka
Saobraćajni	Ustanička 2 Prijedor

$\rho_{\text{fakulteti}(\text{Naziv}, \text{Adresa})}(\text{fakultet})$

fakulteti

Naziv	Adresa
Elektrotehnički	Patre 5 Banjaluka
Saobraćajni	Ustanička 2 Prijedor

18. Objasniti i odgovarajućim primjerom ilustrovati agregatnu operaciju u relacionoj algebri.

Agregatna funkcija uzima (kao ulaz) kolekciju vrijednosti i vraća jednu vrijednost kao rezultat:

- **avg**: prosječna vrijednost
- **min**: minimalna vrijednost
- **max**: maksimalna vrijednost
- **sum**: suma vrijednosti
- **count**: broj vrijednosti

Agregatna operacija je oblika $G_1, G_2, \dots, G_n G_{F_1(A_1), F_2(A_2), \dots, F_m(A_m)}(E)$, gdje je:

- E – bilo koji izraz relacione algebre
- $G_1, \dots, G_n$ – lista atributa po kojima se vrši grupisanje torki ulazne relacije
- $F_1, \dots, F_m$ – agregatne funkcije
- $A_1, \dots, A_m$ – imena atributa

Rezultantna relacija ima $n + m$ atributa.

Relacija koja je rezultat agregatne operacije nema ime, kao ni rezultati agregatnih funkcija (odnosni atributi rezultatne relacije). Za imenovanje se koristi preimenovanje.

nastavnik

JMB	NastZvanje	Plata	JMBsk
1006949100067	vanredni profesor	2000	1907951100012
1002952100005	redovni profesor	2400	null
1907951100012	redovni profesor	2400	null
1804964163303	redovni profesor	2300	1002952100005
1312981163309	docent	1800	1907951100012

$temp \leftarrow G_{avg(Plata)}(nastavnik)$

$\rho_{lista(Prosjeck)}(temp)$

lista

Prosjeck
2180

Primjer:

stud_program

IdSP	NazivFak	NazivSP	Ciklus
RI	Elektrotehnički	Računarstvo i informatika	1
ET	Elektrotehnički	Elektronika i telekomunikacije	2
NF	Prirodno-matematički	Nuklearna fizika	1

Prikazati broj studijskih programa na pojedinim fakultetima

$temp \leftarrow_{NazivFak} G_{count(*)}(stud_program)$

$\rho_{lista(NazivFak, BrojStudProg)}(temp)$

lista

NazivFak	BrojStudProg
Elektrotehnički	2
Prirodno-matematički	1

Prikazati broj studijskih programa na pojedinim fakultetima po ciklusima

$temp \leftarrow_{NazivFak, Ciklus} G_{count(*)}(stud_program)$

$\rho_{lista(NazivFak, Ciklus, BrojSP)}(temp)$

lista

NazivFak	Ciklus	BrojSP
Elektrotehnički	1	1
Elektrotehnički	2	1
Prirodno-matematički	1	1

19. Objasniti i konkretnim primjerom ilustrovati operaciju brisanja torki u relacionoj algebri.

Iskaz za brisanje se specifikuje slično upitu, s tom razlikom da se umjesto prikazivanja korisniku, selektovane torke koriste za brisanje torki iz neke relacije.

Mogu se brisati samo čitave torke – ne mogu se brisati vrijednosti pojedinih atributa.

Brisanje se u relacionoj algebri izvršava kao: $r \leftarrow r - E$, gdje su: r – relacija i E – upit relacione algebre.

Primjer: Modifikovati instancu relacije telefon, tako da se iz relacije briše telefonski broj +38751211408 Elektrotehničkog fakulteta.

telefon

NazivFak	TelefonFak
Elektrotehnički	+38751221820
Elektrotehnički	+38751211408

$temp \leftarrow \sigma_{NazivFak = 'Elektrotehnički' \wedge TelefonFak = '+38751221408'}(telefon)$

$telefon \leftarrow telefon - temp$

telefon

NazivFak	TelefonFak
Elektrotehnički	+38751221820

20. Objasniti i primjerima ilustrovati operacije proširene relacione algebre.

Operacije proširene relacione algebre su:

a. Objasniti i primjerom ilustrovati operaciju generalizovane projekcije.

1. Generalizovana projekcija

Proširuje operaciju projekcije tako da dozvoljava korištenje aritmetičkih izraza/funkcija u projekcionoj listi: $\pi_{F_1, F_2, \dots, F_n}(E)$

E – bilo koji izraz iz relacione algebre

$F_1, F_2, \dots, F_n$ – aritmetički izrazi koji uključuju konstante i attribute šeme koja korespondira izrazu E .

Primjer: Prikazati matične brojeve i iznos plate, izražen u eurima (1 € = 1.95 KM), svih docenata.

nastavnik

JMB	NastZvanje	Plata	JMBsk
1006949100067	vanredni profesor	2000	1907951100012
1002952100005	redovni profesor	2400	null
1907951100012	redovni profesor	2400	null
1804964163303	redovni profesor	2300	1002952100005
1312981163309	docent	1800	1907951100012

$temp \leftarrow \pi_{JMB, Plata / 1.95}(\sigma_{NastZvanje = 'docent'}(nastavnik))$

$\rho_{plate}(JMB, Plata_EUR)(temp)$

plate

JMB	Plata_EUR
1312981163309	923.08

2. Agregatne funkcije i Agregatna operacija

Kao ulaz ima kolekciju vrijednosti i vraća jednu (agregatnu) vrijednost kao rezultat.

Detaljno obrađeno u pitanju 18.

3. Vanjsko (spoljašnje) spajanje

Dodatne operacije spajanja za indikaciju informacija koje nedostaju.

Izvršava operacije spajanja i nakon toga u rezultat dodaje torke iz jedne relacije koje nisu spojene ni sa jednom torkom druge relacije.

Koristi null vrijednosti:

- null označava da je vrijednost (atributa) ili nepoznata ili ne postoji
- Sve aritmetičke operacije koje uključuju null vrijednost, po definiciji kao rezultat vraćaju null vrijednost

Vrste spoljašnjeg (vanjskog) spajanja

- **lijevo spoljašnje spajanje (left outer join)**
 - fakultet $\bowtie$ stud_program
- **desno spoljašnje spajanje (right outer join)**
 - fakultet $\bowtie$ stud_program
- **puno spoljašnje spajanje (full outer join)**
 - fakultet $\bowtie$ stud_program

Primjer: Spajanje relacije fakultet sa relacijom stud_program.

fakultet

NazivFak	AdresaFak
Elektrotehnički	Patre 5 Banjaluka
Saobraćajni	Ustanička 2 Prijedor

stud_program

IdSP	NazivFak	NazivSP
RI	Elektrotehnički	Računarstvo i informatika
ET	Elektrotehnički	Elektronika i telekomunikacije
NF	Prirodno-matematički	Nuklearna fizika

Unutrašnje spajanje (inner join)

fakultet $\bowtie$ stud_program

NazivFak	AdresaFak	IdSP	NazivSP
Elektrotehnički	Patre 5 Banjaluka	RI	Računarstvo i informatika
Elektrotehnički	Patre 5 Banjaluka	ET	Elektronika i telekomunikacije

Lijevo spoljašnje spajanje (left outer join)

fakultet $\bowtie$ stud_program

NazivFak	AdresaFak	IdSP	NazivSP
Elektrotehnički	Patre 5 Banjaluka	RI	Računarstvo i informatika
Elektrotehnički	Patre 5 Banjaluka	ET	Elektronika i telekomunikacije
Saobraćajni	Ustanička 2 Prijedor	null	null

Desno spoljašnje spajanje (right outer join)

fakultet $\bowtie$ stud_program

NazivFak	AdresaFak	IdSP	NazivSP
Elektrotehnički	Patre 5 Banjaluka	RI	Računarstvo i informatika
Elektrotehnički	Patre 5 Banjaluka	ET	Elektronika i telekomunikacije
Prirodno-matematički	null	NF	Nuklearna fizika

Puno spoljašnje spajanje (full outer join)

fakultet $\bowtie$ stud_program

NazivFak	AdresaFak	IdSP	NazivSP
Elektrotehnički	Patre 5 Banjaluka	RI	Računarstvo i informatika
Elektrotehnički	Patre 5 Banjaluka	ET	Elektronika i telekomunikacije
Saobraćajni	Ustanička 2 Prijedor	null	null
Prirodno-matematički	null	NF	Nuklearna fizika

21. Objasniti i konkretnim primjerom ilustrovati operaciju ažuriranja samo određenih torki u relacionoj algebri. Napisati odgovarajuće SQL upite.

Dio koji se ne traži:

Ažuriranje torki obezbjeđuje mehanizam mijenjanja vrijednosti (atributa) u torki bez mijenjanja svih vrijednosti torke.

Ažuriranje se može obezbijediti korištenjem generalizovanog operatora projekcije

$$r \leftarrow \pi_{F_1, F_2, \dots, F_n}(r)$$

Svaki F_i je:

- i-ti atribut relacije r (ako se atribut ne ažurira)
- aritmetički izraz koji uključuje samo konstante i attribute relacije r , i koji specifikuje novu vrijednost atributa (ako se atribut ažurira)

Ovim izrazom se mijenjaju sve torke.

Primjer: Specifikovati izraz relacione algebre kojim se povećava plata svim nastavnicima za 100 KM.

nastavnik

JMB	NastZvanje	Plata
1006949100067	vanredni profesor	2000
1002952100005	redovni profesor	2400
1907951100012	redovni profesor	2400
1804964163303	redovni profesor	2300
1312981163309	docent	1800

$$nastavnik \leftarrow \pi_{JMB, NastZvanje, Plata+100}(nastavnik)$$

Dio koji se traži:

Za ažuriranje samo određenih torki relacije (koje zadovoljavaju određeni uslov P), koristi se specifikacija: $r \leftarrow (\pi_{F_1, \dots, F_n}(\sigma_P(r))) \cup (r - \sigma_P(r))$

Primjer: Specifikovati izraz relacione algebre kojim se povećava plata za 20% nastavniku čiji JMB ima vrijednost 1312981163309.

nastavnik

JMB	NastZvanje	Plata
1006949100067	vanredni profesor	2000
1002952100005	redovni profesor	2400
1907951100012	redovni profesor	2400
1804964163303	redovni profesor	2300
1312981163309	docent	1800

$$temp1 \leftarrow \pi_{JMB, NastZvanje, 1.2*Plata}(\sigma_{JMB=1312981163309}(nastavnik))$$

$$temp2 \leftarrow nastavnik - \sigma_{JMB=1312981163309}(nastavnik)$$

$$nastavnik \leftarrow temp1 \cup temp2$$

```
update nastavnik
set Plata = Plata * 1.2
where JMB = '1312981163309';
```

Relacioni račun

22. Objasniti i odgovarajućim primjerima ilustrovati operacije prirodnog spajanja. Specifikovati odgovarajuće upite u relacionoj algebri, relacionom računu torki, relacionom računu domena i odgovarajuće SQL upite.

Neka su r i s relacije na šemama R i S . Operacija prirodnog spajanja $r \bowtie s$ se definiše na šemi $\text{attrs}(R) \cup \text{attrs}(S)$ koja se dobije na sljedeći način:

- za svaki par torki $t_r \in r$ i $t_s \in s$, ako t_r i t_s imaju iste vrijednosti na svakom od atributa iz $\text{attrs}(R) \cup \text{attrs}(S)$, rezultatu se dodaje torka t , gdje:
 - t ima iste vrijednosti na atributima $\text{attrs}(R)$ kao $t_r \in r$
 - t ima iste vrijednosti na atributima $\text{attrs}(S)$ kao $t_s \in s$

Svrha operacije prirodnog spajanja je spajanje pripadajućih torki iz dvije relacije u jednu torku (tipično, spajanje torki koje su u vezi).

Relacije (tabele) koje se prirodno spajaju moraju imati ista imena atributa (kolona) po kojima se vrši spajanje.

Implikacije: Prije korištenja operacije, mora se znati da postoje zajedničke kolone (sa istim imenima) u tabelama koje se prirodno spajaju. Ako nema zajedničkih atributa (sa istim imenom), onda prirodno spajanje rezultuje Dekartovim proizvodom.

Mogući problemi: Ako dođe do promjene imena zajedničke kolone u tabeli, ako se doda nova kolona, ako dvije tabele imaju kolone koje se često imenuju na identičan način u različitim tabelama.

Primjer: Realizovati prirodno spajanje relacija fakultet i telefon.

fakultet		telefon		fakultet $\bowtie$ telefon		
NazivFak	AdresaFak	NazivFak	TelefonFak	Nazivfak	AdresaFak	TelefonFak
Elektrotehnički	Patre 5 Banjaluka	Elektrotehnički	+38751221820	Elektrotehnički	Patre 5 Banjaluka	+38751221820
Saobraćajni	Ustanička 2 Prijedor	Elektrotehnički	+38751211408	Elektrotehnički	Patre 5 Banjaluka	+38751211408

1. način – relacionala algebra:

$$\pi_{fakultet.NazivFak, AdresaFak, TelefonFak} (\sigma_{fakultet.NazivFak = telefon.NazivFak} (fakultet \times telefon))$$

ili

$$\pi_{fakultet.NazivFak, AdresaFak, TelefonFak} ((fakultet \bowtie_{fakultet.NazivFak = telefon.NazivFak} telefon))$$

ili

$$fakultet \bowtie telefon$$

2. način – relacioni račun torki – TRC:

$$\{ r \mid \exists f \in fakultet (r[NazivFak] = f[NazivFak] \wedge r[AdresaFak] = f[AdresaFak] \\ \wedge \exists t \in telefon (t[NazivFak] = f[NazivFak] \wedge r[TelefonFak] = t[TelefonFak])) \}$$

3. način – relacioni račun domena – DRC:

$$\{ \langle f, a, t \rangle \mid \langle f, a \rangle \in fakultet \wedge \langle f, t \rangle \in telefon \}$$

4. način – SQL upit:

```
select f.NazivFak, f.AdresaFak, t.TelefonFak
from fakultet f
natural join telefon t;
```

23. Objasniti i odgovarajućim primjerom ilustrovati operaciju unije. Specifikovati odgovarajući TRC i DRC upit.

Operacija unije definiše se kao $r \cup s = \{t \mid t \in r \vee t \in s\}$. Da bi operacija bila validana moraju biti ispunjeni sljedeći uslovi:

- r, s moraju imati isti stepen / broj atributa ili arnost
- domeni atributa moraju biti kompatibilni

Primjer: Lista matičnih brojeva osoba koje su nastavnici ili studenti.

nastavnik

JMB	NastZvanje	Plata	JMBsk
1006949100067	vanredni profesor	2000	1907951100012
1002952100005	redovni profesor	2400	null
1907951100012	redovni profesor	2400	null
1804964163303	redovni profesor	2300	1002952100005
1312981163309	docent	1800	1907951100012

student

JMB	AdrStudiranja
1210987100018	Cerska 10 BanjaLuka
0512983100067	Kozarska 23 Banja Luka
0607989100021	Rudarska 102 Banjaluka
2208988105034	Beogradska 2 Banjaluka

1. način – relacionala algebra:

$\pi_{JMB}(nastavnik) \cup \pi_{JMB}(student)$

2. način – TRC:

$\{t \mid \exists s \in student (t[JMB]=s[JMB]) \vee \exists n \in nastavnik (t[JMB]=n[JMB])\}$

3. način – DRC:

$\{ \langle j \rangle \mid \exists a (\langle j, a \rangle \in student) \vee \exists z, p, s (\langle j, z, p, s \rangle \in nastavnik) \}$

Funkcionalne zavisnosti

24. Objasniti zatvaranje skupa funkcionalnih zavisnosti i navesti primjer.

Za dati skup funkcionalnih zavisnosti F, postoje i druge FZ koje proističu iz F (zavisnosti koje su logički implicirane zavisnostima u skupu F) – npr. Ako $A \rightarrow B$ i $B \rightarrow C$, onda možemo zaključiti da $A \rightarrow C$.

Skup svih FZ, logički uključenih sa F, naziva se zatvaranje skupa funkcionalnih zavisnosti F.

Zatvaranje skupa F označava se sa F^+ .

Zatvaranje F^+ može se odrediti primjenom Armstrong-ovih aksioma.

Armstrong-ovi aksiomi:

- Ako $\beta \subseteq \alpha$, onda $\alpha \rightarrow \beta$ (refleksivnost)
- Ako $\alpha \rightarrow \beta$, onda $\gamma \alpha \rightarrow \gamma \beta$ (proširenje)
- Ako $\alpha \rightarrow \beta$ i $\beta \rightarrow \gamma$, onda $\alpha \rightarrow \gamma$ (tranzitivnost)

Armstrongovi aksiomi su kompletni (generišu sve FZ koje vrijede).

Armstrongovi aksiomi su dosljedni (generišu samo FZ koje stvarno postoje i vrijede).

Primjer: Neka je data šema $R = (A, B, C, G, H, I)$ i skup FZ koji vrijedi na šemi R $F = \{ A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H \}$.

Neke FZ iz F^+ su:

- **$A \rightarrow H$**
 - dobijena na osnovu pravila tranzitivnosti $A \rightarrow B$ i $B \rightarrow H$.
- **$AG \rightarrow I$**
 - proširenjem $A \rightarrow C$ sa G , dobije se $AG \rightarrow CG$ i dalje na osnovu tranzitivnosti $CG \rightarrow I$
- **$CG \rightarrow HI$**
 - od $CG \rightarrow H$ i $CG \rightarrow I$ na osnovu pravila „unije“ koje slijedi iz definicije FZ. Ili proširenjem $CG \rightarrow I$ može se zaključiti $CG \rightarrow CGI$, proširenjem $CG \rightarrow H$ sa I može se izvesti $CGI \rightarrow HI$ i dalje primjenom tranzitivnosti dobijamo $CG \rightarrow HI$.

Algoritam za računanje F^+ :

$F^+ = F$

repeat

 za svaku $f \in F^+$

 primjenom pravila refleksivnosti i proširenja formirati nove FZ i dodati ih u F^+

 za svaki par f_1 i $f_2 \in F^+$ (ako se može)

 primjenom pravila tranzitivnosti formirati nove FZ i dodati ih u F^+

until nema promjene F^+

Računanje F^+ može se pojednostaviti primjenom izvedenih pravila (ova pravila se mogu izvesti na osnovu Armstrongovih aksioma):

- **$\alpha \rightarrow \beta \wedge \alpha \rightarrow \gamma \Rightarrow \alpha \rightarrow \beta \gamma$ (unija)**
- **$\alpha \rightarrow \beta \gamma \Rightarrow \alpha \rightarrow \beta \wedge \alpha \rightarrow \gamma$ (dekompozicija)**
- **$\alpha \rightarrow \beta \wedge \gamma \beta \rightarrow \delta \Rightarrow \alpha \gamma \rightarrow \delta$ (pseudotranzitivnost)**

Primjene skupa F^+ :

- **Provjera da li FZ vrijedi na šemi**
 - na osnovu F^+ može se utvrditi da li FZ $\alpha \rightarrow \beta$ vrijedi na datoj relacionoj šemi R , jer ako vrijedi, onda je $\alpha \rightarrow \beta$ saržana u F^+
- **Određivanje kandidatskih ključeva šeme**
 - K je kandidatski ključ na šemu R :
 - ako je K superključ, tj. $K \rightarrow \text{attrs}(R)$ i
 - ako je K minimalni superključ, tj. ne postoji pravi podskup $L \subset K$ takav da $L \rightarrow \text{attrs}(R)$

25. Objasniti i odgovarajućim primjerom ilustrovati određivanje zatvaranja skupa atributa na nekoj relacionoj šemi.

Zatvaranje skupa atributa α označava se sa α^+ .

Zatvaranje skupa atributa α , pod skupom funkcionalnih zavisnosti F , definiše se kao skup atributa koji su funkcionalno zavisni od α , tj. $\alpha \rightarrow \beta \in F^+ \Leftrightarrow \beta \in \alpha^+$.

Algoritam za sračunavanje α^+ :

$\alpha^+ = \alpha$

do {

$\alpha^+_{preth} = \alpha^+$

for each $\beta \rightarrow \gamma \in F$

if $\beta \subseteq \alpha^+$ **then** $\alpha^+ = \alpha^+ \cup \gamma$

}

while ($\alpha^+ \neq \alpha^+_{preth}$)

Primjer: Neka je data šema $R = (A, B, C, D, E)$ i skup FZ koji vrijedi na šemi R $F = \{ B \rightarrow CE, C \rightarrow D, E \rightarrow A \}$. Odrediti skup atributa funkcionalno zavisan od A, B, C, D i E respektivno.

• A^+

- Inicijalno: $A^+ = \{ A \}$.
- Pošto u skupu FZ ne postoji ni jedna funkcionalna zavisnost koja na lijevoj strani ima atribut koji je podskup od A^+ , neće doći do promjene.
- Rezultat je: $A^+ = \{ A \}$.

• B^+

- Inicijalno: $B^+ = \{ B \}$.
- $B \rightarrow CE \Rightarrow B^+ = B^+ \cup \{ CE \} = \{ BCE \}$
- $C \rightarrow D \Rightarrow B^+ = B^+ \cup \{ D \} = \{ BCED \}$
- $E \rightarrow A \Rightarrow B^+ = B^+ \cup \{ A \} = \{ BCEDA \}$
- Rezultat je: $B^+ = \{ ABCDE \}$

• C^+

- Analogno prethodnom imamo: $C^+ = \{ CD \}$

• D^+

- Analogno prethodnom imamo: $D^+ = \{ D \}$

• E^+

- Analogno prethodnom imamo: $E^+ = \{ EA \}$

Dekompozicija

26. Objasniti i odgovarajućim primjerom ilustrovati dekompoziciju relacije šeme bez gubitka informacija.

Dekompozicija relacije šeme R na skup relacionih šema $\{R_1, R_2, \dots, R_n\}$ je dekompozicija bez gubitka informacija ako je za svaku legalnu relaciju $r(R)$:

$$r = \pi_{R_1}(r) \bowtie \dots \bowtie \pi_{R_n}(r),$$

tj. prirodnim spajanjem relacija dobijenih dekompozicijom ne generišu se lažne torke, nego samo torke polazne relacije r .

Neka je R relaciona šema na kojoj vrijedi skup funkcionalnih zavisnosti F i neka je $\{R_1, R_2\}$ binarna dekompozicija šeme R . Dekompozicija je bez gubitka informacija ako i samo ako u F^+ postoji barem jedna od sljedećih FZ:

$$R_1 \cap R_2 \rightarrow R_1 \quad \text{ili} \quad R_1 \cap R_2 \rightarrow R_2$$

odnosno:

$$\text{attrs}(R_1) \subseteq (\text{attrs}(R_1) \cap \text{attrs}(R_2))^+ \quad \text{ili}$$

$$\text{attrs}(R_2) \subseteq (\text{attrs}(R_1) \cap \text{attrs}(R_2))^+$$

Svi atributi originalne šeme R moraju se pojaviti u dekompoziciji $(R_1, \dots, R_n)$:

$$R = R_1 \cup \dots \cup R_n.$$

Uslov za lossless-join dekompoziciju: zajednički atributi šema R_1 i R_2 predstavljaju superključ na bar jednoj od šema R_1, R_2 – **modifikovana Heath-ova teorema**.

Primjer: Neka je data šema: $R = (A, B, C)$ i skup FZ koji vrijedi na šemi R : $F = \{A \rightarrow B, B \rightarrow C\}$.

Razdvojimo R na dvije relacije šeme $R_1 = (A, B)$ i $R_2 = (B, C)$. Jeste dekompozicija bez gubitka informacija, jer je $R_1 \cap R_2 = \{B\}$ i $B \rightarrow BC$ (tj. $R_1 \cap R_2 \rightarrow R_2$). (B je superključ na R_2)

27. Druga normalna forma. Odgovarajućim primjerom ilustrovati relacionu šemu koja ne ispunjava uslove 2NF, a zatim je dekomponovati u odgovarajuće šeme koje ispunjavaju uslove 2NF.

Relaciona šema je u 2NF:

- ako je 1NF i
- ako je svaki njen neprimarni atribut potpuno i funkcionalno zavisan od PK (i od svakog kandidatskog ključa) te šeme.

Atribut A je neprimarni atribut relacije šeme R ako nije dio primarnog (kandidatskog) ključa relacije šeme R , tj. $A \notin PK(KK)$.

Skup atributa β je potpuno funkcionalno zavisan od skupa atributa α , ako $\alpha \rightarrow \beta$ i ne postoji nijedan pravi podskup $\alpha' \subset \alpha$, takav da $\alpha' \rightarrow \beta$.

Skup atributa β je parcijalno funkcionalno zavisan od skupa atributa α , ako $\alpha \rightarrow \beta$ i postoji pravi podskup $\alpha' \subset \alpha$, takav da $\alpha' \rightarrow \beta$.

Relaciona šema nije u 2NF ako postoji parcijalna funkcionalna zavisnost nekog neprimarnog atributa od PK te šeme.

Ako je PK relacije šeme prost (samo jedan atribut), ne postoji mogućnost parcijalne funkcionalne zavisnosti neprimarnih atributa, ta šema je u 2NF.

Primjer: Uzmimo relaciju šemu definisanu kao STUDENT_SP(JMB, IdSP, Ime, GodinaUpisa, BrojIndeksa, SemestarSP, NazivSP, Ciklus).

Možemo vidjeti iz FZ $\{JMB, IdSP\} \rightarrow NazivSP$, ali i $IdSP \rightarrow NazivSP$, tako da uslov za 2NF nije ispunjen.

Tako da vršimo dekompoziciju (eliminišemo parcijalne FZ) da bismo dobili 2NF na sljedeće relacije šeme:

STUDENT(JMB, Ime)

STUD_PROGRAM(IdSP, NazivSP, Ciklus)

UPISAN_NA(JMB, IdSP, GodinaUpisa, BrojIndeksa, SemestarSP)

28. Treća normalna forma. Odgovarajućim primjerom ilustrovati relacionu šemu koja ne ispunjava uslove 3NF, a zatim je dekomponovati u odgovarajuće šeme koje ispunjavaju uslove 3NF.

Klasična definicija 3NF: Relaciona šema je u 3NF ako je u 2NF i ako nijedan neprimarni atribut nije tranzitivno zavisn od PK te šeme.

Neprimarni atribut A šeme R je tranzitivno zavisn od PK α ako postoji skup atributa β takav da $\alpha \rightarrow \beta$ i $\beta \rightarrow A$, pri čemu β nije kandidatski ključ šeme R.

Relaciona šema R je u 3NF ako za svaku FZ $\alpha \rightarrow \beta$ u F^+ vrijedi jedan od sljedećih uslova:

- $\alpha \rightarrow \beta$ je trivijalna FZ (tj. $\beta \in \alpha$)
- α je superključ na R
- svaki atribut A u $\beta - \alpha$ je sadržan u nekom od kandidatskih ključeva šeme R.

Ako je relacija u BCNF ona je i u 3NF (pošto u BCNF jedan od prva dva uslova moraju vrijediti za sve FZ).

Primjer: Uzmimo relaciju oblika SP_FAK(IdSP, NazivSP, Ciklus, NazivFak, AdresaFak). Ova relacija jeste u 2NF, ali nije u 3NF, jer ima tranzitivnu FZ – $NazivFak \rightarrow AdresaFak$. Potrebno je da eliminišemo tranzitivne FZ da bismo dobili 3NF i to postizemo dekompozicijom na dvije relacije šeme.

FAKULTET(NazivFak, AdresaFak)

STUD_PROGRAM(IdSP, NazivSP, Ciklus, NazivFak)

29. BCNF. Odgovarajućim primjerom ilustrovati relacionu šemu koja ne ispunjava uslove BCNF, a zatim je dekomponovati u odgovarajuće šeme koje ispunjavaju uslove BCNF.

Relaciona šema je u BCNF (Boyce-Codd-ovoj normalnoj formi) s obzirom na skup funkcionalnih zavisnosti F, ako za sve FZ u F^+ oblika $\alpha \rightarrow \beta$ ($\alpha \subseteq R$ i $\beta \subseteq R$) vrijedi:

- $\alpha \rightarrow \beta$ je trivijalna FZ (tj. $\beta \subseteq \alpha$) ili

- α je superključ na šemi R.

Primjer: Neka je data relaciona šema $R = (A, B, C)$ i skup funkcionalnih zavisnosti koji vrijedi na R $F = \{ A \rightarrow B, B \rightarrow C \}$.

$A^+ = \{ ABC \}$, $B^+ = \{ BC \}$, $C^+ = \{ C \}$ – superključ = primarni ključ = $\{ A \}$.

$A \rightarrow B$ zadovoljava zahtjeve BCNF (A je superključ).

$B \rightarrow C$ ne zadovoljava zahtjeve BCNF ($B \rightarrow C$ nije trivijalna FZ, niti je B superključ)

Zaključak: **R nije u BCNF!**

Radimo dekompoziciju šeme R na:

$R1 = (A, B)$ i $R2 = (B, C)$

$R1: F1 = \{ A \rightarrow B \}$, A je superključ na $R1$ – jeste BCNF

$R2: F2 = \{ B \rightarrow C \}$, B je superključ na $R2$ – jeste BCNF

$R1 \cap R2 = \{ B \}$ i B je superključ na $R2$ – dekompozicija bez gubitka podataka

$R1: F1 = \{ A \rightarrow B \}$, $R2: F2 = \{ B \rightarrow C \}$ – očuvane FZ

Zaključak: **Dobili smo BCNF!**

Nije moguće uvijek dobiti BCNF dekompoziciju koja ima očuvane FZ!

Primjer: $R = (J, K, L)$ $F = \{ JK \rightarrow L, L \rightarrow K \}$

R nije u BCNF, jer FZ $L \rightarrow K$ ne zadovoljava uslove za BCNF (L nije superključ).

Na R imaju dva kandidatska ključa: **JK** i **JL**

- $JK \rightarrow J$ (triv. FZ), $JK \rightarrow K$ (triv. FZ), $JK \rightarrow L \Rightarrow (JK)^+ = \{ JKL \} \Rightarrow JK$ je kandidatski ključ
- $JL \rightarrow J$ (triv. FZ), $JL \rightarrow L$ (triv. FZ), $JL \rightarrow K$ (jer $L \rightarrow K$) $\Rightarrow (JL)^+ = \{ JKL \} \Rightarrow JL$ je kandidatski ključ

Dekompozicija šeme R: $R1 = (L, K)$ i $R2 = (J, L)$ – imaju po dva atributa i nikako ne mogu očuvati FZ $JK \rightarrow L$ koja uključuje sva 3 atributa.

Zaključak: Dekompozicija R u BCNF nije očuvala FZ ($JK \rightarrow L$).

30. Neka su date šeme OSOBA(jmb, ime, prezime), STUDENT(jmb, indeks). Specifikovati TRC upit kojim se iz relacija o(OSOBA) i s(STUDNET) dobija lista svih studenata koja sadrži sljedeće podatke za svakog studenta: jmb, indeks, ime, prezime.

31. Objasniti operaciju prirodnog spajanja na primjeru relacija fakultet i telefon (iz prethodnog zadatka), a zatim specifikovati korespondentne upite u relacionoj algebri, relacionom računu torki, relacionom računu domena, kao i odgovarajući SQL upit.

32. Objasniti i primjerima ilustrovati operacije agregatne funkcije i agregatnu operaciju. Za svaki upit u relacionoj algebri napisati korespondentni SQL upit.

33. Objasniti i primjerima ilustrovati operacije spoljašnjeg (i lijevog) spajanja, a zatim specificovati korespondentne upite u relacionoj algebri i odgovarajuće SQL upite.